

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 2: Acceso a datos en aplicaciones web del entorno servidor

Tema 4: Lenguajes de marcas de uso común en el lado servidor

Origen e historia de los lenguajes de marcas. El estándar XML

Características de XML

Estructura de XML

Estándares basados en XML

Análisis XML

Uso de XML en el intercambio de información

Lenguajes de marcas de uso común en el lado servidor

1 Origen e historia de los lenguajes de marcas. El estándar XML.

Los lenguajes de marcas surgieron para estructurar y organizar datos en documentos electrónicos. XML (Extensible Markup Language) es un estándar desarrollado por el W3C para definir datos estructurados de manera legible tanto para humanos como para máquinas.

Características de XML

1 Partes de un documento XML: marcas, elementos, atributos, etc.

- **Marcas (tags):** Delimitan los elementos dentro del documento.
- **Elementos:** Contienen datos y pueden anidarse dentro de otros elementos.
- **Atributos:** Información adicional dentro de las etiquetas.

Ejemplo:

```
xml
<persona edad="30">
  <nombre>Juan</nombre>
  <apellido>Pérez</apellido>
</persona>
```

2 Sintaxis y semántica de documentos XML: documentos válidos y bien formados.

- **Bien formados:** Siguen la sintaxis correcta (etiquetas cerradas, anidamiento correcto, etc.).
- **Válidos:** Cumplen con una DTD o un esquema XML.

Ejemplo de documento bien formado:

```
xml
<libro>
  <titulo>XML en profundidad</titulo>
  <autor>María Gómez</autor>
</libro>
```

Ejemplo de documento no válido (según DTD):

```
xml
<libro>
  <titulo>XML en profundidad</titulo>
</libro>
```

(Si la DTD requiere un elemento `<autor>`, el documento no es válido.)

Estructura de XML.

1 Esquemas XML: DTD y XML Schema.

- **DTD (Document Type Definition):** Define la estructura de un documento XML con reglas estrictas.
- **XML Schema (XSD):** Más potente que DTD, permite definir tipos de datos.

Ejemplo de DTD:

```
xml
<!DOCTYPE persona [
    <!ELEMENT persona (nombre, edad)>
    <!ELEMENT nombre (#PCDATA)>
    <!ELEMENT edad (#PCDATA)>
]>
```

Ejemplo de XML Schema:

```
xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="persona">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="nombre" type="xs:string"/>
                <xs:element name="edad" type="xs:int"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

2 Hojas de estilo XML: el estándar XSLT y XSL.

- **XSL (Extensible Stylesheet Language):** Conjunto de tecnologías para transformar y presentar XML.
- **XSLT (XSL Transformations):** Permite transformar XML en HTML, PDF, etc.

Ejemplo de XSLT para transformar XML en HTML:

```
xml
<xsl:template match="persona">
  <html>
    <body>
      <h2>Información de la persona</h2>
      <p>Nombre: <xsl:value-of select="nombre"/></p>
    </body>
  </html>
</xsl:template>
```

3 Enlaces: XLL.

- **XLL (XML Linking Language):** Define enlaces dentro de documentos XML.

Ejemplo de XLink:

```
xml
<enlace xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="https://example.com">Visitar</enlace>
```

Estándares basados en XML.

1 Presentación de página: XHTML.

XHTML es una versión de HTML basada en XML con sintaxis estricta.

Ejemplo:

```
xml
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Ejemplo XHTML</title>
</head>
<body>
  <p>Hola, mundo.</p>
</body>
</html>
```

2 Selección de elementos XML: XPath y XQuery.

- **XPath:** Lenguaje para navegar por documentos XML.
- **XQuery:** Lenguaje para consultar XML, similar a SQL.

Ejemplo de XPath para seleccionar un nodo:

```
xpath  
/persona/nombre
```

Ejemplo de XQuery:

```
xquery  
for $p in doc("personas.xml")//persona  
where $p/edad > 30  
return $p/nombre
```

3 Firma electrónica y cifrado.

- **XML-Signature:** Protocolo para firmar documentos XML.
- **XML-Encryption:** Para cifrar contenido XML.

Análisis XML.

1 Herramientas y utilidades de análisis.

- **DOM (Document Object Model):** Modelo jerárquico para manipular XML.
- **SAX (Simple API for XML):** Procesa XML en flujo, sin cargar todo el documento en memoria.
- **Herramientas:** XMLSpy, Notepad++, lxml (Python), DOMDocument (PHP).

2 Programación de análisis XML mediante lenguajes en servidor.

Ejemplo en PHP usando DOM:

```
php
$xml = new DOMDocument();
$xml->load("archivo.xml");
echo $xml->getElementsByTagName("nombre")->item(0)->nodeValue;
```

Uso de XML en el intercambio de información.

1 Codificación de parámetros.

Usar XML para enviar datos estructurados en servicios web.

Ejemplo:

```
xml
<solicitud>
  <usuario>Juan</usuario>
  <accion>compra</accion>
</solicitud>
```

2 Ficheros de configuración basados en XML.

Muchos sistemas almacenan configuraciones en XML (ej. `config.xml`).

Ejemplo. Leer un XML desde un archivo (Configuración de Base de Datos):

Supongamos que tenemos un archivo config.xml con la siguiente estructura:

```
xml
<configuracion>
  <base_datos>
    <host>localhost</host>
    <usuario>root</usuario>
    <password>1234</password>
    <nombre>mi_basedatos</nombre>
  </base_datos>
</configuracion>
```

Para leer los datos en PHP:

```
php
$xml = simplexml_load_file('config.xml') or die("Error al cargar XML");

echo "Host: " . $xml->base_datos->host . "<br>";
echo "Usuario: " . $xml->base_datos->usuario . "<br>";
echo "Contraseña: " . $xml->base_datos->password . "<br>";
echo "Nombre BD: " . $xml->base_datos->nombre;
```

Explicación:

- simplexml_load_file('config.xml') carga el archivo XML en un objeto.
- Se accede a los valores como si fueran propiedades (->).

Ejemplo: Modificar un valor en el XML y guardarlo

Si queremos cambiar el usuario de la base de datos, podemos hacerlo así:

```
php

$xml->base_datos->usuario = 'nuevo_usuario';

// Guardamos los cambios en el archivo
$xml->asXML('config.xml');

echo "Usuario actualizado con éxito.";
```

Explicación:

- Se accede a la propiedad y se le asigna un nuevo valor.
- asXML('config.xml') guarda los cambios en el archivo.

Ejemplo: Añadir un nuevo nodo al XML

Si queremos agregar un nuevo campo, por ejemplo, el puerto de la base de datos:

php

```
$xml->base_datos->addChild('puerto', '3306');
```

```
// Guardamos los cambios
```

```
$xml->asXML('config.xml');
```

```
echo "Campo 'puerto' agregado con éxito.";
```

Explicación:

- addChild('puerto', '3306') agrega un nuevo elemento <puerto>3306</puerto>.

Ejemplo: Crear un nuevo archivo XML desde PHP

Podemos generar un archivo XML completamente desde PHP:

```
php
$xml = new SimpleXMLElement('<configuracion/>');

$base_datos = $xml->addChild('base_datos');
$base_datos->addChild('host', 'localhost');
$base_datos->addChild('usuario', 'root');
$base_datos->addChild('password', '1234');
$base_datos->addChild('nombre', 'mi_basedatos');

$xml->asXML('nueva_config.xml');

echo "Archivo XML creado con éxito.";
```

Explicación:

- Se crea un objeto SimpleXMLElement.
- Se agregan nodos con addChild().
- asXML('nueva_config.xml') guarda el archivo.

Ejercicio: Gestión de Productos en XML con PHP

Objetivo

Crear un sistema que permita:

- ✓ Leer productos desde un archivo XML.
- ✓ Agregar nuevos productos mediante un formulario en PHP.
- ✓ Mostrar los productos en una tabla HTML.
- ✓ Guardar los cambios en el archivo XML.

1. Archivo productos.xml (Datos iniciales)

Creas un archivo llamado productos.xml con la siguiente estructura:

xml

```
<productos>
  <producto>
    <id>1</id>
    <nombre>Teclado mecánico</nombre>
    <precio>79.99</precio>
    <categoria>Accesorios</categoria>
  </producto>
  <producto>
    <id>2</id>
    <nombre>Ratón inalámbrico</nombre>
    <precio>49.99</precio>
    <categoria>Accesorios</categoria>
  </producto>
</productos>
```



2. Archivo index.php (Leer y mostrar productos)

Este script cargará los productos desde el XML y los mostrará en una tabla HTML.

```
php
<?php
// Cargar el XML
$xml = simplexml_load_file('productos.xml') or die("Error al cargar XML");

echo "<h2>Lista de Productos</h2>";
echo "<table border='1'>
<tr>
    <th>ID</th>
    <th>Nombre</th>
    <th>Precio (€)</th>
    <th>Categoría</th>
</tr>";

// Recorrer y mostrar los productos
foreach ($xml->producto as $producto) {
    echo "<tr>
        <td>{$producto->id}</td>
        <td>{$producto->nombre}</td>
        <td>{$producto->precio}</td>
        <td>{$producto->categoria}</td>
    </tr>";
}

echo "</table>";
?>

<!-- Formulario para agregar productos -->
<h2>Agregar Producto</h2>
```

```
<form action="agregar.php" method="post">
  Nombre: <input type="text" name="nombre" required><br>
  Precio: <input type="number" name="precio" step="0.01" required><br>
  Categoría: <input type="text" name="categoria" required><br>
  <input type="submit" value="Agregar">
</form>
```



3. Archivo agregar.php (Añadir productos al XML)

Este script guardará los nuevos productos en el archivo XML.

```
php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Cargar el archivo XML
    $xml = simplexml_load_file('productos.xml') or die("Error al cargar XML");

    // Crear un nuevo nodo producto
    $nuevo_producto = $xml->addChild('producto');

    // Asignar valores del formulario
    $nuevo_id = count($xml->producto) + 1; // Genera un nuevo ID autoincrementado
    $nuevo_producto->addChild('id', $nuevo_id);
    $nuevo_producto->addChild('nombre', $_POST['nombre']);
    $nuevo_producto->addChild('precio', $_POST['precio']);
    $nuevo_producto->addChild('categoria', $_POST['categoria']);

    // Guardar cambios en el archivo XML
    $xml->asXML('productos.xml');

    echo "Producto agregado con éxito. <a href='index.php'>Volver</a>";
}
?>
```

¿Cómo probarlo?

- 1 Coloca los archivos productos.xml, index.php y agregar.php en tu servidor local (XAMPP o similar).
 - 2 Abre index.php en el navegador para ver la lista de productos.
 - 3 Rellena el formulario y presiona “Agregar”.
 - 4 El nuevo producto se guardará en productos.xml y aparecerá en la lista.
-

Ampliaciones posibles

- ◆ Añadir opción para eliminar productos.
- ◆ Modificar productos existentes.
- ◆ Validar entradas del formulario.